Your UnEmbedding Matrix is Secretly a Feature Lens for Text Embeddings

Songhao Wu, Zhongxin Chen, Yuxuan Liu, Heng Cui, Cong Li, Rui Yan, qwen.qwen3.5-122b

ABSTRACT

Large language models exhibit impressive zero-shot capabilities across a wide range of downstream tasks. However, they struggle to function as off-the-shelf embedding models, leading to suboptimal performance on massive text embedding benchmarks. In this paper, we identify a potential cause underlying this deficiency. Our motivation stems from an unexpected observation: text embeddings tend to align with frequent but uninformative tokens when projected onto the vocabulary space. We argue that this excessive expression of high-frequency tokens suppresses the model's ability to capture nuanced semantics. To address this, we introduce EmbedFilter, a simple linear transformation designed to refine text embeddings derived from LLMs directly. Specifically, we uncover that the unembedding matrix within LLMs encodes a latent space that is actively writing these frequent tokens into embedding space. By filtering out this subspace, EmbedFilter suppress the influence of high-frequency tokens, thereby enhancing semantic representations. As a compelling byproduct, this enables an inherent dimensionality reduction, lowering index storage and speedup retrieval while fully preserving the refined embedding quality. Our experiments across multiple LLM backbones demonstrate that LLMs equipped with EmbedFilter achieve superior zero-shot downstream performance even with significantly reduced embedding dimensions. We hope our findings provide deeper insights into the mechanisms of LLM-based representations and inspire more principled designs to improve text embeddings training. Our code is available at https://github.com/CentreChen/EmbFilter.

About this document

This is a **Reviewed Preprint**: a third-party paper that llmXive has *auto-reviewed* (with its LLM review panel) but *never modified*. The original manuscript follows this cover page **exactly as published** by its authors — llmXive re-typesets nothing and claims no authorship.
Provenance. Ingested by llmXive on 2026-07-02 — scraped from arXiv; via llmXive dashboard, GitHub issue #290; submitted by github-actions[bot].
Source. https://arxiv.org/abs/2606.07502
About llmXive. llmXive is an automated platform for open scientific discovery. Its specialist LLM agents advance their own ideas from brainstorm to peer-reviewed paper, and they also review

notable third-party papers — drawn from the most-downloaded papers on Hugging Face and from submissions by human contributors. Every submitted paper is reviewed by the LLM panel *and* used to seed a new llmXive follow-up study. This paper is one such third-party work: llmXive reviewed it but did not write it. Learn more at `https://github.com/ContextLab/llmXive`.

Automated review. See the accompanying `peer-review-llmxive.pdf` for the full reviewer report.

llmXive follow-up. A separate llmXive study building on this work: `PROJ-880-llmxive-follow-up-extending-your-unembed`.

Your UnEmbedding Matrix is Secretly a Feature Lens for Text Embeddings

Songhao Wu*
Gaoling School of Artificial Intelligence, Renmin University of China
Beijing, China
songhaowu@ruc.edu.cn

Zhongxin Chen*
Gaoling School of Artificial Intelligence, Renmin University of China
Beijing, China
chenzhongxin@ruc.edu.cn

Yuxuan Liu
Gaoling School of Artificial Intelligence, Renmin University of China
Beijing, China
yuxuanliu@ruc.edu.cn

Heng Cui
Lenovo Group Limited
Beijing, China
cuiheng3@lenovo.com

Cong Li
Lenovo Group Limited
Beijing, China
licong17@lenovo.com

Rui Yan†
Wuhan University
Wuhan, China
rui.yan@whu.edu.cn

Abstract

Large language models exhibit impressive zero-shot capabilities across a wide range of downstream tasks. However, they struggle to function as off-the-shelf embedding models, leading to suboptimal performance on massive text embedding benchmarks. In this paper, we identify a potential cause underlying this deficiency. Our motivation stems from an unexpected observation: text embeddings tend to align with frequent but uninformative tokens when projected onto the vocabulary space. We argue that this excessive expression of high-frequency tokens suppresses the model's ability to capture nuanced semantics. To address this, we introduce EmbedFilter, a simple linear transformation designed to refine text embeddings derived from LLMs directly. Specifically, we uncover that the unembedding matrix within LLMs encodes a latent space that is actively writing these frequent tokens into embedding space. By filtering out this subspace, EmbedFilter suppress the influence of high-frequency tokens, thereby enhancing semantic representations. As a compelling byproduct, this enables an inherent dimensionality reduction, lowering index storage and speedup retrieval while fully preserving the refined embedding quality. Our experiments across multiple LLM backbones demonstrate that LLMs equipped with EmbedFilter achieve superior zero-shot downstream performance even with significantly reduced embedding dimensions. We hope our findings provide deeper insights into the mechanisms of LLM-based representations and inspire more principled designs to improve text embeddings training. Our code is available at https://github.com/CentreChen/EmbFilter.

CCS Concepts

• **Information systems** → *Language models*; *Novelty in information retrieval*.

Keywords

Zero-shot Text Embedding, Large Language Model, Mechanistic Interpretation

ACM Reference Format:
Songhao Wu, Zhongxin Chen, Yuxuan Liu, Heng Cui, Cong Li, and Rui Yan. 2018. Your UnEmbedding Matrix is Secretly a Feature Lens for Text Embeddings. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 10 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Large language models (LLMs) have made significant strides in recent years, demonstrating impressive performance across a wide range of tasks [8, 12, 28]. The emergence of zero-shot learning ability helps LLMs address unseen tasks effectively without any additional fine-tuning [15]. However, recent studies highlight a persistent performance gap of LLMs when deployed as zero-shot text embedding models [1, 14, 19]. This deficiency hinders their adoption for text embedding tasks and raises concerns regarding their full efficacy as generalist models in real-world applications.

To bridge this gap, researchers have explored various attempts to better elicit semantic information from LLMs. Prompt-engineering methods have been proposed to help extract text embeddings directly from LLMs [14, 17, 26, 30]. These approaches are well motivated; however, their improvements are modest and highly sensitive to the choice of the prompt, leading to inconsistent performance across different setups. Existing approaches are primarily heuristic and fail to resolve the bottleneck that limits LLMs' ability to capture semantics. In this paper, we move beyond previous heuristic efforts and seek to provide a mechanistic interpretation for LLMs' suboptimal performance in text embedding tasks. Specifically, we identify an unexpected representation collapse: when projected onto the vocabulary space, raw text embeddings from LLMs tend to align with high-frequency tokens that are semantically irrelevant. Equipped with the Logit Lens tool [2], we find that frequent

*These authors contributed equally. Songhao Wu discovered the core phenomenon, provided the core implementation and led the writing. Zhongxin Chen refined the code, conducted the experiments and provided Songhao Wu with valuable insights.
†Corresponding Author.

Conference acronym 'XX, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

but uninformative tokens disproportionately dominate the highest decoding probabilities of these text embeddings. This suggests that these hidden representations are biased toward common vocabulary tokens, regardless of the input semantics[1]. As shown in Figure 1, this phenomenon is observed across different language model families, indicating a universal pattern inherent to LLMs.

We extend our analysis to uncover the underlying drivers of this representation collapse. Prior studies [9, 18] have established that text embeddings are *anisotropic*: they are confined to a narrow cone rather than being uniformly distributed in the embedding space. We hypothesize that the centroid of this narrow region corresponds to an "average" token, which Lv et al. [20] describe as the frequency-weighted average embedding over the training corpus. This perspective provides a mechanistic rationale for the atypical patterns observed in Logit Lens analyses. Raw embeddings from LLMs are pulled toward this commonality region, overshadowing their unique semantic features. By suppressing the contribution of these "average" components, we can mitigate the anisotropy problem and unmask the true semantic representations within LLMs.

We seek to pinpoint the hidden contributor that steer text embeddings towards the "average" token representation. To this end, we apply Logit Spectroscopy [6] to a reverse-engineered "average" token, and uncover a latent subspace, which is actively writing these frequent tokens into the embedding space. We refer to this subspace as the *"edge spectrum"* space, as it is spanned by the right singular vectors with the smallest and largest singular values — those positioned at the ends of the spectrum. We find that when the projection of the "average" token onto this subspace is truncated, the logits of these frequent tokens are significantly disrupted. Section 3 delves into the discovery of the edge spectrum, providing a detailed account of its identification

Leveraging this insight, we show that this subspace can be effectively filtered out via a simple linear transformation, which we term EmbedFilter. This transformation is encoded within the parameters of the unembedding matrix and is readily accessible without further training. Our evaluations across a diverse suite of downstream tasks demonstrate that EmbedFilter acts as a potent post-processing enhancement, delivering steady incremental gains atop existing zero-shot text embedding baselines. EmbedFilter exhibits strong robustness across various backbone models and experimental configurations while incurring minimal computational overhead. Beyond performance gains, EmbedFilter naturally lends itself to dimensionality reduction as a distance-preserving transformation. This reduction lowers indexing overhead and speeds up retrieval, facilitating the practical deployment of LLMs.

To sum up, the contributions of this paper are threefold.

(1) We identify the LLM unembedding matrix as a previously overlooked feature lens to analyze the embedding space. We reveal that this matrix encodes a latent subspace corresponding to an "average" token and limits the embedding capabilities of LLMs. We provide an mechanism interpretation that clarifies both the origins and impact of this phenomenon.

(2) We introduce EmbedFilter, a simple linear transformation that improves the zero-shot text embedding performance of LLMs. As an efficient post-processing technique, EmbedFilter achieves up to a 14.1% improvement on MTEB without any training overhead. Extensive evaluations across diverse experimental setups further demonstrate its broad applicability.

(3) We demonstrate that EmbedFilter acts as a distance-preserving transformation and enable embedding dimensionality reduction. This leads to faster retrieval and lower storage requirements, thereby facilitating the practical deployment of LLMs in large-scale text embedding applications.

[1]For readers unfamiliar with Logit Lens, please refer to Section 2 for further details.

2 Background

To establish the background for EmbedFilter we first review the fundamentals of embedding extraction and introduce the mechanistic interpretability tools used throughout our analysis.

2.1 Text Embedding Paradigm

We first formulate the standard process of LLM-based text embedding extraction. Our objective is to transform sentence $\boldsymbol{X}$ into a dense vector $\boldsymbol{h} \in \mathbb{R}^d$, such that the similarity between these vectors can reflect their semantic similarity. Given an input sentence $\boldsymbol{X} = [x_1, x_2, \ldots, x_L]$, its embedding $\boldsymbol{h}$ is obtained by passing $\boldsymbol{X}$ through an LLM backbone, followed by a pooling strategy P:

$$\boldsymbol{h} = \mathrm{P}\left(\mathrm{LLM}\left(\left[x_1, x_2, \ldots, x_L\right]\right)\right),$$

where P aggregates the final layer outputs from LLM into a d-dimensional representation $\boldsymbol{h}$. Typically, the unembedding matrix is conceptually designed to map these hidden states back to the vocabulary space for token prediction. We contend that this module has been overlooked in the context of traditional text embedding extraction and can be exploited to enhance embeddings qualities.

2.2 Text Embeddings with Prompt Engineering

Many studies have explored improving the performance of LLMs on text embedding tasks through prompt engineering. Here, we provide a brief overview of two well-established baselines:

PromptEOL [14] finds that a "one word limitation" template can help better condense semantics into the hidden state, thereby enhancing the representation of LLM-derived embeddings.

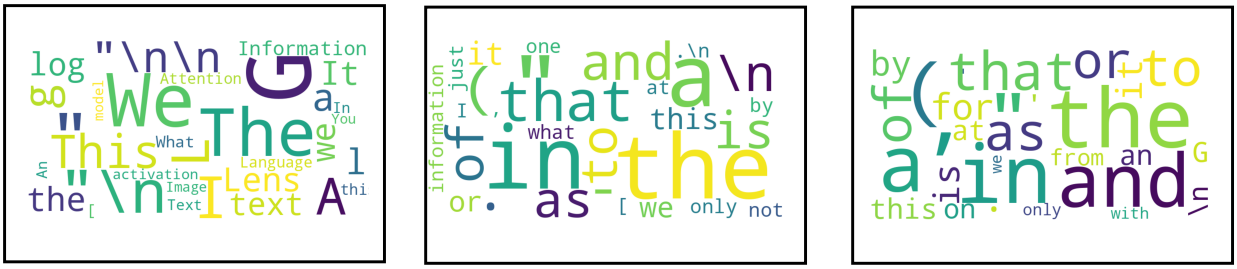
ECHO [26] suggests that causal attention in LLMs is a bottleneck, as earlier tokens cannot access future context. To mitigate this, they duplicate the input and extract embeddings from the second occurrence, incurring overhead from the increased input size.

More sophisticated prompt-engineering methods have been proposed [17, 30]; however, these often necessitate intricate pipeline designs and incur substantial computational overhead. While our primary experiments focus on the aforementioned baselines, we provide a broader discussion and evaluation of these more complex strategies in our supplementary analysis.

2.3 Mechanistic Interpretability Tools

We provide an overview of two interpretability tools — Logit Lens [2] and Logit Spectroscopy [6] — which facilitate the identification of edge spectrum subspace and inspire the design of EmbedFilter.

Logit Lens [2] represents a cornerstone of mechanistic interpretability research. Its central premise is to project a model's intermediate representations directly into the vocabulary space. By



(a) Qwen-2.5-05B **(b) Llama-3.1-8B-Instruct** **(c) Mistral-7B-Instruct-V0.3**

Figure 1: Logit Lens applied to text embeddings from three LLM backbones. Word clouds show the top-aligned tokens with the highest decoding probabilities, which are primarily high-frequency yet semantically uninformative. The input text, encoded by the text embeddings, is given as: *"We call this a 'lens' because it is one way of extracting information from GPT's internal activations. I imagine there is other information present in the activations that cannot be understood by looking at logits over tokens. The logit lens show us some of what is going on, not all of it."* This corresponds to the official notation of the logit lens.

analyzing the resulting changes in these logits, researchers can discern how specific intermediate activations shape the final predictions, thereby gaining insights into the model's internal processing logic. Building on this framework, Nie et al. [23] apply the Logit Lens tool to text embeddings and find that these embeddings can align with certain keywords from the input texts.

To further dissect the semantic properties of different embedding subspaces, **Logit Spectroscopy** [6] extends Logit Lens by projecting intermediate representations onto spectral components of model's weight matrices. Let $\boldsymbol{W_U}$ be the unembedding matrix of the LLM. Its singular value decomposition can be formulated as:

$$\boldsymbol{W_U} = \boldsymbol{U}\,\Sigma\,\boldsymbol{V}^\top,$$

where $\boldsymbol{W_U} \in \mathbb{R}^{|\mathcal{V}|\times d}$, with d representing the hidden-state dimension and $|\mathcal{V}|$ the vocabulary size. For an arbitrary dimension $i \in \{0,\ldots,d-1\}$, Logit Spectroscopy introduces a filter $\boldsymbol{\Psi_i}$ that removes the projection of $\boldsymbol{h}$ onto the i-th right singular vector of $\boldsymbol{V}$. Formally, this transformation is defined as:

$$\boldsymbol{\Psi_i} = \boldsymbol{I} - \boldsymbol{V}_{[i]}\,\boldsymbol{V}_{[i]}^\top.$$

This operation facilitates the spectral analysis of an LLM's intermediate representations, enabling researchers to measure the contribution of hidden states within different spectral subspaces to the final output. Section 3 details how we leverage these tools to identify the "edge spectrum" subspace.

3 Discovery of Edge Spectrum Subspace

3.1 Motivation

In this section, we present the preliminaries analyses that motivate the development of EmbedFilter. Our investigation is driven by an observed correlation between two key insights:

(1) Raw text embeddings from LLMs are typically anisotropic [18, 27]. These embeddings are concentrated in a narrow subspace, making them excessively similar to one another;

(2) LLM-derived embeddings often align with high-frequency tokens that carry little semantics.

These insights lead us to reasonably infer that the narrow subspace is responsible for encoding frequent tokens. Consequently, we seek to isolate this subspace and mitigate its impact, thereby alleviating the anisotropy problem in text embedding tasks. To accomplish this, we first reverse-engineer a "centroid" hidden state representing the "average" token. We then perform Logit Spectroscopy on this "average" token, revealing that the edge spectrum subspace drives the emergence of high-frequency tokens. We present the technical details of this discovery below.

3.2 Reverse-Engineering of the Average Token

We leverage the unembedding matrix, together with word frequencies from training corpus, to reverse-engineer the "average" token.

3.2.1 Experimental Setup. We evaluate a diverse set of models, ranging from Qwen-2.5 [28] (0.5B) to Mistral-v0.3-Instruct [13] (7B) and Llama-3.1 Instruct [12] (8B). By spanning multiple scales and model families, we aim to ensure the universality of our findings.

Since pretraining datasets for these LLMs are not disclosed, we approximate their true word frequency distribution $\boldsymbol{p}$ by sampling tokens from open-source corpora. Specifically, we select the RedPajama [33] dataset as our evaluation corpus. Parallel experiments on alternative corpora produce identical results. The resulting empirical statistics, denoted as $\hat{\boldsymbol{p}}$, serve as a robust proxy for distribution $\boldsymbol{p}$ and are adopted throughout the following experiments.

3.2.2 Reverse-Engineering. We outline the practical steps for reverse-engineering the "average" token. For a standard inference step, the unembedding matrix is used to compute the probability distribution over the next token. Formally, this prediction step is given by:

$$\boldsymbol{q} = \mathrm{Softmax}\left(\boldsymbol{h}\boldsymbol{W}_{\mathcal{U}}^\top\right),$$

where the probability of an arbitrary token i is given by:

$$\boldsymbol{q}_i = \exp(\boldsymbol{w}_i) \;/\; \textstyle\sum_{j=1}^{|\mathcal{V}|} \exp(\boldsymbol{w}_j).$$

Given this, the logit $\boldsymbol{w}_i$ of the i-th token is denoted as:

$$\boldsymbol{w}_i \;=\; \log(\boldsymbol{q}_i) \;+\; \log \sum\nolimits_{j=1}^{|\mathcal{V}|} e^{\boldsymbol{w}_j},$$

where the second term is a shared bias across all logits, which we redefine as $\boldsymbol{b}$. The logits for decoding $\boldsymbol{h}$ is reformulated as:

$$\boldsymbol{h}\boldsymbol{W}_{\mathcal{U}}^{\top} \;=\; \log(\boldsymbol{q}) \;+\; \boldsymbol{b}.$$

By denoting the Moore–Penrose pseudo-inverse [24] of $\boldsymbol{W}_{\mathcal{U}}^{\top}$ as $\boldsymbol{W}_{\mathcal{U}}^{+}$, we can further rewrite the preceding formula as:

$$\boldsymbol{h} \;=\; (\log(\boldsymbol{q}) \;+\; \boldsymbol{b})\;\boldsymbol{W}_{\mathcal{U}}^{+}.$$

We substitute the observed word frequencies $\hat{\boldsymbol{p}}$ and interpret $\hat{\boldsymbol{h}}$ as the "average" token representation over the training corpus. Formally, the average token embedding is defined as:

$$\hat{\boldsymbol{h}} \;=\; \log(\hat{\boldsymbol{p}})\;\boldsymbol{W}_{\mathcal{U}}^{+},$$

where the bias term $\boldsymbol{b}$ is omitted for analytical simplicity, since it does not alter the fundamental spectral properties.

3.2.3 Logit Spectroscopy into Average Token. Having established the theoretical foundation of Logit Spectroscopy, we now detail its application to the average token. For each dimension $i \in \{0, \ldots, d-1\}$, we apply a filter $\boldsymbol{\Psi}_i$ to remove the projection of $\hat{\boldsymbol{h}}$ onto the subspace, resulting in the perturbed representation $\widetilde{\boldsymbol{h}}^{(i)}$, defined as:

$$\widetilde{\boldsymbol{h}}^{(i)} \;=\; \hat{\boldsymbol{h}}\left(\boldsymbol{I} \;-\; \boldsymbol{V}_{[i]}\boldsymbol{V}_{[i]}^{\top}\right).$$

We analyze the logit shifts between $\boldsymbol{h}$ and $\widetilde{\boldsymbol{h}}^{(i)}$ for the k most frequent tokens in the training corpus. Let $\mathcal{V}^{+}$ denote this subset of frequent tokens, formally defined as $\mathcal{V}^{+} = \{j \mid j \in \text{argtopk}(\hat{\boldsymbol{p}})\}$. The impact of the filtering operation is then quantified by the cumulative logit differences across these tokens, which is given as:

$$\Delta\pi^{(i)} \;=\; \frac{\sum_{j\in\mathcal{V}^{+}} \left|\widetilde{w}_j^{(i)} - \hat{w}_j\right|}{\sum_{j\in\mathcal{V}^{+}} \left|\hat{w}_j\right|},$$

where $\hat{\boldsymbol{w}}_j$ represents the original logit of the j-th token, and $\widetilde{\boldsymbol{w}}_j^{(i)}$ denotes the logit after filtering out the subspace spanned by the i-th right singular vector of $\boldsymbol{W}_{\mathcal{U}}$. A higher value of $\Delta\pi^{(i)}$ indicates that the i-th singular subspace exerts a more pronounced influence on the representation of high-frequency tokens.

Figure 2 presents the $\Delta\pi$ values when setting $k = 100$. As shown, the $\Delta\pi$ values are significantly larger at the edges of the spectrum, suggesting that the subspaces corresponding to the edge spectrum of LLMs are primarily responsible for encoding high-frequency tokens. This specific spectral region is precisely what we aimed to identify. As demonstrated in the following sections, filtering out this edge spectrum not only suppresses the over-representation of "average" tokens but also enhances the quality of LLM-derived text embeddings. For comparison, Figure 4 visualizes the influence of different spectral subspaces on the representation of infrequent and randomly sampled tokens. Notably, the logit differences for

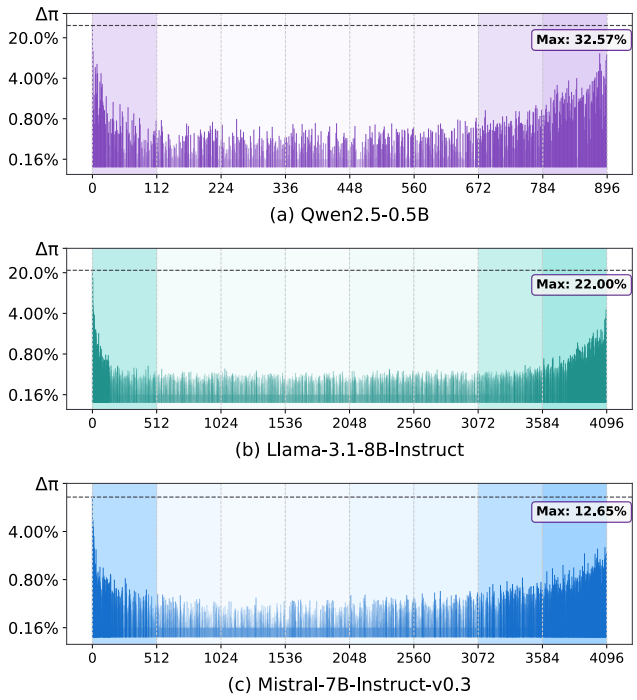


Figure 2: $\Delta\pi$ distribution for Qwen, Llama and Mistral.

infrequent and random tokens exhibit significantly lower sensitivity to the edge spectrum than those for frequent tokens.

4 Text embedding with EmbedFilter

Building on our preliminary insights, we propose EmbedFilter, a simple linear transformation to filter out the edge spectrum subspace. This section provides an overview of the EmbedFilter workflow. Additionally, we present a dimensionality reduction approach based on EmbedFilter to highlight its efficiency.

4.1 Methodology Formulation of EmbedFilter.

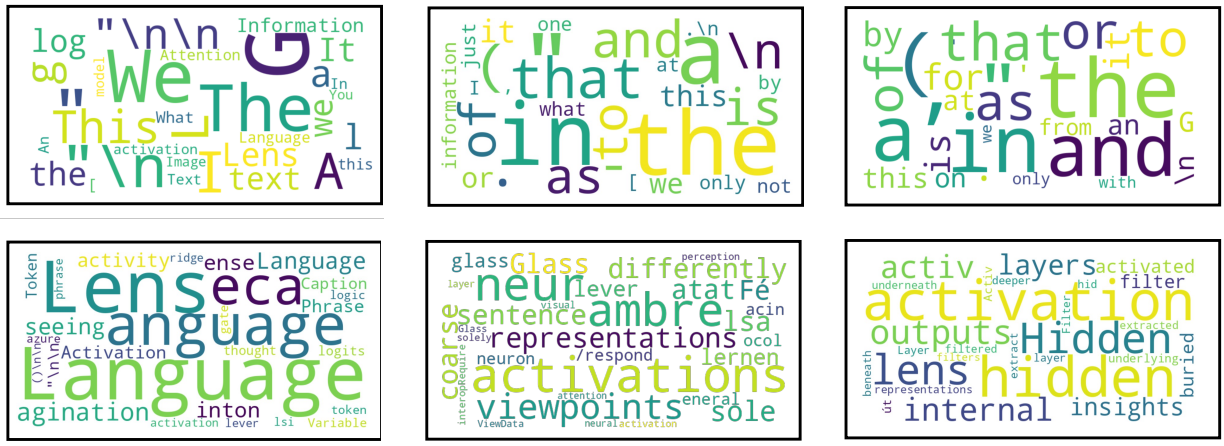
We introduce the Bulk Spectrum Transformation ($\boldsymbol{\Phi}_r$), to filter out the edge spectrum space of raw LLM-derived text embeddings. By excluding the right singular vectors associated with both the largest and smallest singular values, we construct $\boldsymbol{\Phi}_r$ from the remaining mid-range singular components. We hypothesize that this "bulk" of the spectrum suppresses the influence of non-semantic tokens, thereby enabling a more effective capture of core semantics within the embedding space. Formally, the matrix $\boldsymbol{\Phi}_r$ is defined as:

$$\boldsymbol{\Phi}_{\tau} \;=\; \boldsymbol{V}[l_{\tau} : r_{\tau}]\;\boldsymbol{V}[l_{\tau} : r_{\tau}]^{\top},$$

where τ is a predefined filtering ratio, with l_{τ} and r_{τ} denoting the start and end indices of the columns. We use this transformation to post-process the existing embeddings $\{\boldsymbol{e}_i\}_{i=1}^{N}$, and map them into refined representations $\widetilde{\boldsymbol{e}_i}$ optimized for downstream tasks:

$$\widetilde{\boldsymbol{e}_i} \;=\; \boldsymbol{e}_i\,\boldsymbol{\Phi}_{\boldsymbol{\tau}}^{\top}.$$

Model	Top 6 Tokens From Logit Lens					
Qwen	G	We	"	The	"\n	I
+ EmbFilter	Language	Lens	anguage	eca	agination	_Language
Llama	_the	,	_a	_"	_in	_that
+ EmbFilter	_activations	_neur	ambre	_viewpoints	_representations	sole
Mistral	,	the	in	a	(	and
+ EmbFilter	hidden	activation	hidden	Hidden	lens	activ



(a) Qwen-2.5-05B (b) Llama-3.1-8B-Instruct (c) Mistral-7B-Instruct-V0.3

Figure 3: Re-running logit lens analysis in Section 1 with text embeddings refined by EmbedFilter. Top-6 tokens from logit lens are displayed, with colored entries indicate tokens that have literal connections with the input text. EmbedFilter suppresses the expression of frequent tokens and enhances the semantic richness of text embeddings.

This transformation safely filters out the edge spectrum space while preserving the components in the bulk spectrum. Further implementation details can be found in our code repository. We then use EmbedFilter to refine the text embeddings and re-run the Logit Lens analysis, with the corresponding before-and-after comparisons presented in Figure 3.

4.2 Dimensionality Reduction

Moreover, we observe that text embeddings refined by EmbedFilter facilitate dimensionality reduction for free. Recall that $\boldsymbol{V}$ represents the right singular vectors of $\boldsymbol{W_U}$. Since $\boldsymbol{V}$ is an orthogonal matrix, it constitutes, by definition, a distance-preserving transformation. Given that, for any $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^d$, we have the identity:

$$\|\boldsymbol{x}\,\boldsymbol{\Phi}_{\boldsymbol{\tau}}^{\top} - \boldsymbol{y}\,\boldsymbol{\Phi}_{\boldsymbol{\tau}}^{\top}\|_2 \;=\; \|\boldsymbol{x}\,\boldsymbol{V}[l_\tau : r_\tau] - \boldsymbol{y}\,\boldsymbol{V}[l_\tau : r_\tau]\|_2. \quad (1)$$

Given the properties presented in Equation 1, we can replace $\boldsymbol{\Phi}_{\boldsymbol{r}}^{\top}$ with $\boldsymbol{V}\,[l_\tau : r_\tau]$, which causes no theoretical difference in similarity measurement. For readers unfamiliar with these properties, we also provide a simple proof of Equation 1 in the Appendix B.

By invoking this identity transformation, we substantially reduce the hidden size of the raw text embeddings. This reduction translates to reduced index storage overhead and faster retrieval speeds, as it minimizes both memory bandwidth bottlenecks and distance computation complexity during search. Our experimental results in Section 5 demonstrate that this approach successfully achieves significant dimensionality reduction while maintaining or even exceeding downstream task performance, thereby achieving improvements in both efficiency and effectiveness simultaneously.

5 Experiment

5.1 General Setup.

We evaluate EmbedFilter's effectiveness on the MTEB benchmark [22], which includes standard downstream applications for text embeddings such as Semantic Textual Similarity (STS), Classification (Class.), Clustering (Cluster.), and Retrieval (Retr.). We build our evaluation framework upon the official MTEB implementation and report the standard metrics for each task. Due to limited computational resources, we evaluate a subset of the retrieval tasks, following the protocols in [1, 19]. Detailed descriptions of the experimental configurations and subset selection can be found in Appendix A. We evaluate EmbedFilter across three backbone LLMs (Qwen, Llama, and Mistral), ensuring comprehensive coverage of mainstream architectures and model scales.

Table 1: Performance of EmbedFilter across MTEB tasks. τ controls dimensionality reduction, scaling the output dimensionality to $1/\tau$ of the original size. Colored entries highlight improvements over the vanilla baseline, while bold text mark the best results within each setup. Parenthetical values indicate the performance gain of EmbedFilter compared to its baseline.

	STS.	Class.	Cluster.	PairClass.	Rerank.	Retr.	Sum.	Avg. ↑
Num. Datasets (→)	**10**	**12**	**11**	**3**	**4**	**8**	**1**	**49**
Qwen2.5-0.5B								
PromptEOL	63.04	69.20	34.91	55.15	49.33	27.31	27.30	50.07
+ EmbFilter ($\tau = 2$)	**69.48**	**70.32**	**39.20**	**64.72**	**51.28**	**34.73**	27.12	**54.57 (+9.0%)**
+ EmbFilter ($\tau = 4$)	68.57	68.92	38.24	64.54	50.62	32.85	27.67	53.47 (+6.8%)
+ EmbFilter ($\tau = 8$)	68.03	66.07	35.50	63.57	49.70	29.82	**28.37**	51.43 (+2.7%)
ECHO	63.98	64.86	30.16	55.54	42.80	18.15	22.78	46.03
+ EmbFilter ($\tau = 2$)	**70.77**	**67.37**	**36.94**	**66.35**	**46.59**	**29.65**	29.73	**52.55 (+14.1%)**
+ EmbFilter ($\tau = 4$)	69.64	65.59	36.17	65.33	46.40	28.61	**31.65**	51.50 (+11.9%)
+ EmbFilter ($\tau = 8$)	68.81	61.91	34.80	63.57	46.13	25.42	29.79	49.43 (+7.4%)
Llama-3.1-8B-Instruct								
PromptEOL	75.19	73.39	39.30	64.22	53.67	25.45	25.49	55.13
+ EmbFilter ($\tau = 2$)	**76.66**	**73.78**	**40.67**	**66.64**	**54.68**	29.69	27.39	**56.79 (+3.0%)**
+ EmbFilter ($\tau = 4$)	76.63	73.73	40.57	66.63	54.65	**29.86**	27.51	56.78 (+3.0%)
+ EmbFilter ($\tau = 8$)	76.33	73.10	40.32	66.41	54.41	29.70	**27.93**	56.46 (+2.4%)
ECHO	70.43	68.80	38.89	66.98	49.26	30.14	25.41	53.52
+ EmbFilter ($\tau = 2$)	**74.41**	**69.77**	**42.64**	**73.98**	**53.15**	**39.21**	28.46	**57.70 (+7.8%)**
+ EmbFilter ($\tau = 4$)	74.20	69.13	42.28	73.94	53.07	38.64	**28.97**	57.32 (+7.1%)
+ EmbFilter ($\tau = 8$)	74.05	67.50	41.88	73.76	52.75	37.75	28.58	56.61 (+5.8%)
Mistral-7B-Instruct-v0.3								
PromptEOL	64.15	**71.26**	33.40	58.51	48.10	20.91	24.72	49.47
+ EmbFilter ($\tau = 2$)	66.59	71.17	36.16	62.07	49.63	24.59	24.33	51.50 (+4.1%)
+ EmbFilter ($\tau = 4$)	67.55	70.92	37.41	63.29	50.11	**25.97**	24.66	52.26 (+5.6%)
+ EmbFilter ($\tau = 8$)	**68.11**	70.07	**38.04**	**63.67**	**50.20**	25.92	**25.79**	**52.35 (+5.8%)**
ECHO	72.81	71.60	32.42	71.48	47.56	28.37	31.49	53.21
+ EmbFilter ($\tau = 2$)	74.66	**71.79**	36.14	**74.96**	51.66	35.03	31.23	56.10 (+5.4%)
+ EmbFilter ($\tau = 4$)	74.85	71.05	**37.07**	74.91	**51.87**	**35.49**	31.14	**56.25 (+5.7%)**
+ EmbFilter ($\tau = 8$)	**74.86**	70.00	36.92	74.29	51.71	34.91	**31.56**	55.82 (+4.9%)

5.2 Main Results on MTEB.

Table 1 presents the main experimental results of EmbedFilter on MTEB, configured with both PromptEOL and ECHO. Specifically, we analyze EmbedFilter's performance with different filtering ratios to assess its sensitivity. We have the following observations:

(1) EmbedFilter demonstrates notable improvements across all experimental setups, providing strong evidence of its effectiveness and robustness. Specifically, EmbedFilter delivers remarkable enhancements over the baselines, achieving up to a 14% increase in MTEB overall performance. These performance gains are maintained even when the output embedding size is reduced to only 1/8 of its original dimension. Furthermore, EmbedFilter consistently achieves superior overall performance across all evaluated setups, whereas the prompt-engineering methods exhibits performance fluctuations. This underscores the generalization capability of EmbedFilter and highlights its potential for integration with a broader spectrum of LLMs.

(2) EmbedFilter introduces only a lightweight linear transformation module, ensuring negligible overhead during the post-processing of large-scale text embeddings. Additional experimental results in Table 2 and 3, demonstrate that EmbedFilter remains highly effective even when integrated into sophisticated prompt-engineering pipelines, such as MetaEOL [17] and GenEOL [30]. Unlike these complex frameworks — which requires iterative calls to powerful commercial LLMs or the aggregation of multiple embeddings for a single sentence — EmbedFilter bypasses the heavy computational overhead of these complex extraction framework design, leading to superior downstream performance with higher efficiency.

Table 2: Performance of EmbedFilter on MTEB via MetaEOL prompting.

	STS.	Class.	Cluster.	PairClass.	Rerank.	Retr.	Sum.	Avg. ↑
MetaEOL (Qwen)	67.15	71.43	33.44	69.09	50.26	28.17	29.28	52.23
+ EmbFilter ($\tau = 2$)	**71.27**	**71.69**	**37.19**	**72.28**	**51.65**	**34.58**	**31.83**	**55.39 (+6.1%)**
+ EmbFilter ($\tau = 4$)	70.54	70.33	36.00	71.57	50.82	33.82	30.80	54.38 (+4.1%)
MetaEOL (Llama)	71.23	74.89	41.31	72.50	52.44	32.16	29.87	56.73
+ EmbFilter ($\tau = 2$)	**73.68**	**75.53**	**43.15**	**75.08**	53.60	**36.60**	30.42	**58.79 (+3.6%)**
+ EmbFilter ($\tau = 4$)	73.59	75.47	42.89	75.02	**53.61**	36.41	**30.62**	58.67 (+3.6%)

Table 3: Performance of EmbedFilter on STS tasks under the GenEOL framework.

	STS12	STS13	STS14	STS15	STS16	STS17	STS22	SICK-R	STSB	BIOSSES	Avg. ↑
GenEOL	71.36	84.89	77.29	80.94	81.17	84.21	67.72	78.19	79.23	72.27	77.73
+ EmbFilter ($\tau = 2$)	71.28	85.19	77.92	81.60	81.87	86.08	68.51	78.89	80.14	76.38	**78.39**
+ EmbFilter ($\tau = 2$)	70.38	84.81	77.20	81.00	81.22	85.15	66.99	78.31	79.31	76.42	78.08

5.3 The Effect of Filtering Ratio τ

As aforementioned, we introduce a hyperparameter τ to represent the filtering ratio in EmbedFilter. Consequently, the dimensionality of text embeddings is reduced to $1/\tau$ of the original size. This reduction is critical, as it scales down the index storage to $1/\tau$ of its previous occupation and theoretically result in $\tau\times$ speedup in similarity computation. A larger value of τ indicates lower memory usage and faster retrieval speeds, which is especially beneficial in real-world applications. Based on this, we analyze the impact of τ on the performance of EmbedFilterÀs shown in Table 1, EmbedFilter consistently delivers improvement acorss different choices of τ. Remarkably, it retains competitive, and in some cases, superior performance on MTEB tasks, even at a high filtering ratio of $\tau = 8$.

Large language models typically have larger hidden sizes, leading to increased storage and computational costs when deployed as embedding models. By incorporating EmbedFilter, LLMs can attain improved downstream performance with smaller representation dimensions. We present the dimensionality reduction performance of Llama-3.1-8B-Instruct with EmbedFilter in Table 4. With the aid of EmbedFilter, zero-shot LLMs can outperform established, well-trained baselines from the pre-LLM era, such as SimCSE [11] and coCondensor [10], while utilizing smaller representation dimensions. This advancement enables the direct deployment of LLMs as embedding models in low-resource scenarios.

5.4 Ablation Studies of Filtering Strategies

We evaluate various configurations of our filtering strategies to verify the effectiveness of the EmbedFilter design. Specifically, we conduct a detailed ablation analysis using Qwen2.5-0.5B with PromptEOL and a dimensionality reduction ratio of $\tau = 2$. The results across these different experimental setups are reported in Table 5. We can draw the following conclusions:

(1) The improvement of EmbedFilter does not stem from a simple reduction in the dimensionality of text embeddings. For configuration ①, we truncate the first half of the dimensions from the original text embeddings, following the Matryoshka setup [16]. In configuration ②, we randomly choose half of the dimensions from the original d-dimensional vector to form the reduced embeddings. Configuration ① and ② have fewer vector dimensions but still underperform the vanilla PromptEOL. Therefore, we contend that the improvements brought by EmbedFilter are not merely due to the reduction in the dimensionality.

(2) EmbedFilter provides the most effective strategy for subspace filtering. Our comparisons include configuration ③ through ⑤, where we selectively filter the right singular subspaces associated with the largest (Dominant), smallest (Secondary), and intermediate (Bulk) singular values, respectively. Compared to these variants, EmbedFilter achieves the best downstream performance. Notably, configuration ⑤ — the inverse operation of EmbedFilter — obtains the poorest results. Moreover, we find that Configuration ④ significantly outperforms ⑤. This finding is in line with the $\Delta\pi$ distribution shown in Figure 2, where the secondary subspace exhibits a greater tendency to encode frequent tokens than the dominant subspace. We leave the exploration of optimal strategies for filtering the asymmetric edge spectrum subspace to future work.

(3) EmbedFilter is remarkably effective, nearly reaching the theoretical upper bound of our framework's potential. In configuration ⑥, we identify singular vectors with the largest $\Delta\pi^{(i)}$ based on our analysis in Section 3 and filter out the corresponding subspace. We regard this configuration as the theoretical upper bound of EmbedFilter's capability. As shown in Table 5, EmbedFilter performs competitively with configuration ⑥ while requiring no task-specific calibration and being significantly simpler to implement.

5.5 Comparison between EmbedFilterand Embedding Calibration Baselines

We also compare EmbedFilter with established embedding calibration baselines. These methods typically derive text embeddings from a calibration dataset and propose improvements based on the

Table 4: Dimensionality reduction performance of Llama with EmbedFilter on MTEB.

	Dim.	STS	Class.	Cluster.	PairClass.	Rerank.	Retr.	Sum.	Avg. (↑)
SimCSE-BERT-sup	**768**	79.12	67.32	33.43	74.89	47.53	26.34	31.17	53.54
coCondenser-msmarco	**768**	76.47	64.71	37.64	81.74	51.85	35.14	29.50	55.48
PromptEOL	**4096**	70.43	68.80	38.89	66.98	49.26	30.14	25.41	53.52
+ EmbFilter ($\tau = 8$)	**4096**	76.33	73.10	40.32	66.41	54.41	29.70	**27.93**	**56.46**
ECHO	**4096**	70.43	68.80	38.89	66.98	49.26	30.14	25.41	53.52
+EmbFilter ($\tau = 8$)	**512**	74.05	67.50	41.88	73.76	52.75	37.75	28.58	**56.61**
MetaEOL	**4096**	71.23	74.89	41.31	72.50	52.44	32.16	29.87	56.73
+ EmbFilter ($\tau = 8$)	**512**	73.49	74.74	42.47	74.55	53.39	35.89	30.72	**58.25**

Table 5: Ablation studies of the filtering strategies. Best results are in bold.

	STS	Class.	Cluster.	PairClass.	Rerank.	Retr.	Sum.	Avg.
PromptEOL	63.04	69.20	34.91	55.15	49.33	27.31	27.30	50.07
+ EmbFilter	**69.48**	70.32	**39.20**	64.72	**51.28**	**34.73**	27.12	**54.57**
① Truncation	62.56	68.54	33.52	54.81	48.93	25.34	27.67	49.13
② Random	63.27	68.29	34.15	54.55	48.81	25.03	27.03	49.27
③ Dominant	60.34	66.97	33.25	51.18	48.13	22.89	27.00	47.53
④ Secondary	67.74	**70.49**	36.28	62.71	50.27	33.17	**29.26**	53.19
⑤ Bulk	59.92	67.22	32.08	50.71	47.83	22.47	27.46	47.13
⑥ Optimal	68.52	69.97	38.63	**65.03**	51.03	34.68	28.67	54.19

Table 6: MTEB results for EmbedFilter and whitening. Best results are highlighted in bold.

	Dim.	STS	Class.	Cluster.	PairClass.	Rerank.	Retr.	Sum.	Avg. (↑)
PromptEOL	**896**	63.04	69.20	34.91	55.15	49.33	27.31	**27.30**	50.07
EmbFilter	**448**	**69.48**	**70.32**	**39.20**	64.72	**51.28**	**34.73**	27.12	**54.57 (+9.0%)**
whitening	**448**	67.18	69.62	36.03	**67.67**	50.56	32.92	26.98	53.04 (+5.9%)

resulting statistical properties. A representative baseline is Bert-whitening [27], which addresses the anisotropic issue by applying a whitening operation to the text embeddings. Notably, BERT-whitening also facilitates dimensionality reduction consequently.

Given this, we compare EmbedFilter and whitening on Qwen and set $\tau = 2$. We follow the experimental setups from [27], and report the results with supervision of NLI dataset [5]. Their results on MTEB are presented in Table 6. While whitening helps improve the performance, EmbedFilter still outperforms it without the supervision of any calibration data. We argue that the unembedding matrix of LLMs captures valuable statistical features during the pretraining phase that have been previously overlooked. We did not include this method as a baseline in Table 1, as its reliance on calibration data would lead to an unfair comparison with EmbedFilter.

While EmbedFilter is primarily heuristic, we also provide a whitening perspective to help understand. In effect, it can be interpreted as a whitening-like operation within bulk spectral space:

$$\widetilde{\boldsymbol{e}}_i = \boldsymbol{e}_i \boldsymbol{\Phi}_r^\top = \sum_{j=l_\tau}^{r_\tau} \alpha_j \boldsymbol{v}_j, \quad \text{where } \alpha_j = \text{proj}_{\boldsymbol{v}_j} \boldsymbol{e}_i.$$

Text embeddings exhibit more uniform projections onto directions associated with mid-range singular values, providing a relatively isotropic subspace for free. We leave a deeper investigation into the underlying mechanisms of this phenomenon to future work, and we hope this perspective will inspire readers and inform future advancements in text embedding training.

6 Conclusion

In this paper, we investigate the suboptimal zero-shot performance of LLMs on text embedding tasks and provide a mechanistic interpretation. Through an analysis of the model's unembedding matrix, we discover the edge spectrum space, which is responsible for encoding high-frequency tokens into the embedding space. Motivated by this finding, we introduce EmbedFilter, a simple linear transformation to filter out this spectrum space. Our experiments across multiple LLM backbones demonstrate that applying EmbedFilter leads to superior zero-shot improvements on text embedding tasks. Crucially, we also find that this filtering design implicitly reduces the effective dimensionality of the embeddings, thereby lowering index storage overhead and accelerating retrieval. We hope our findings provide insights and inspire more principled designs to improve text embeddings training.

Acknowledgment

This work is supported by Lenovo Group. We thank Ang Lv for his writing suggestions and guidance during the rebuttal phase. We are also grateful to Yuhan Liu and Yankai Lin for providing computational resources and API access. Additionally, we sincerely acknowledge the anonymous KDD reviewers for their constructive comments and questions, which have greatly improved this work.

References

[1] Parishad BehnamGhader, Vaibhav Adlakha, Marius Mosbach, Dzmitry Bahdanau, Nicolas Chapados, and Siva Reddy. 2024. LLM2Vec: Large Language Models Are Secretly Powerful Text Encoders. arXiv:2404.05961 [cs.CL] https://arxiv.org/abs/2404.05961

[2] Nora Belrose, Zach Furman, Logan Smith, Jason Wu, Brian Ge, Alisa Trakhtenberg, Misha Shah, and Jacob Gurney. 2023. Eliciting Latent Predictions from Transformers with the Tuned Lens. *arXiv preprint arXiv:2303.08112* (2023).

[3] Alexander Bondarenko, Maik Fröbe, Meriem Beloucif, Lukas Gienapp, Yamen Ajjour, Alexander Panchenko, Chris Biemann, Benno Stein, Henning Wachsmuth, Martin Potthast, and Matthias Hagen. 2020. Overview of Touché 2020: Argument Retrieval: Extended Abstract. In *Experimental IR Meets Multilinguality, Multimodality, and Interaction: 11th International Conference of the CLEF Association, CLEF 2020, Thessaloniki, Greece, September 22–25, 2020, Proceedings* (Thessaloniki, Greece). Springer-Verlag, Berlin, Heidelberg, 384–395. doi:10.1007/978-3-030-58219-7_26

[4] Vera Boteva, Demian Gholipour, Artem Sokolov, and Stefan Riezler. 2016. A Full-Text Learning to Rank Dataset for Medical Information Retrieval. In *Proceedings of the European Conference on Information Retrieval (ECIR)* (Padova, Italy). Springer.

[5] Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. 2015. A large annotated corpus for learning natural language inference. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, Lluís Màrquez, Chris Callison-Burch, and Jian Su (Eds.). Association for Computational Linguistics, Lisbon, Portugal, 632–642. doi:10.18653/v1/D15-1075

[6] Nicola Cancedda. 2024. Spectral Filters, Dark Signals, and Attention Sinks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 4792–4808. doi:10.18653/v1/2024.acl-long.263

[7] Arman Cohan, Sergey Feldman, Iz Beltagy, Doug Downey, and Daniel S. Weld. 2020. SPECTER: Document-level Representation Learning using Citation-informed Transformers. In *ACL*.

[8] DeepSeek-AI. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence.

[9] Kawin Ethayarajh. 2019. How Contextual are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.). Association for Computational Linguistics, Hong Kong, China, 55–65. doi:10.18653/v1/D19-1006

[10] Luyu Gao and Jamie Callan. 2022. Unsupervised Corpus Aware Language Model Pre-training for Dense Passage Retrieval. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 2843–2853. doi:10.18653/v1/2022.acl-long.203

[11] Tianyu Gao, Xingcheng Yao, and Danqi Chen. 2021. SimCSE: Simple Contrastive Learning of Sentence Embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 6894–6910. doi:10.18653/v1/2021.emnlp-main.552

[12] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[13] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL] https://arxiv.org/abs/2310.06825

[14] Ting Jiang, Shaohan Huang, Zhongzhi Luan, Deqing Wang, and Fuzhen Zhuang. 2024. Scaling Sentence Embeddings with Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 3182–3196. doi:10.18653/v1/2024.findings-emnlp.181

[15] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).

[16] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham Kakade, Prateek Jain, et al. 2022. Matryoshka representation learning. *Advances in Neural Information Processing Systems* 35 (2022), 30233–30249.

[17] Yibin Lei, Di Wu, Tianyi Zhou, Tao Shen, Yu Cao, Chongyang Tao, and Andrew Yates. 2024. Meta-Task Prompting Elicits Embeddings from Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 10141–10157. doi:10.18653/v1/2024.acl-long.546

[18] Bohan Li, Hao Zhou, Junxian He, Mingxuan Wang, Yiming Yang, and Lei Li. 2020. On the Sentence Embeddings from Pre-trained Language Models. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 9119–9130.

[19] Ziyue Li and Tianyi Zhou. 2025. Your Mixture-of-Experts LLM Is Secretly an Embedding Model for Free. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=eFGQ97z5Cd

[20] Ang Lv, Yuhan Chen, Kaiyi Zhang, Yulong Wang, Lifeng Liu, Ji-Rong Wen, Jian Xie, and Rui Yan. 2024. Interpreting Key Mechanisms of Factual Recall in Transformer-Based Language Models. arXiv:2403.19521 [cs.CL] https://arxiv.org/abs/2403.19521

[21] Macedo Maia, Siegfried Handschuh, André Freitas, Brian Davis, Ross McDermott, Manel Zarrouk, and Alexandra Balahur. 2018. WWW'18 Open Challenge: Financial Opinion Mining and Question Answering. (2018), 1941–1942.

[22] Niklas Muennighoff, Nouamane Tazi, Loic Magne, and Nils Reimers. 2023. MTEB: Massive Text Embedding Benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, Andreas Vlachos and Isabelle Augenstein (Eds.). Association for Computational Linguistics, Dubrovnik, Croatia, 2014–2037. doi:10.18653/v1/2023.eacl-main.148

[23] Zhijie Nie, Richong Zhang, and Zhanyu Wu. 2025. A Text is Worth Several Tokens: Text Embedding from LLMs Secretly Aligns Well with The Key Tokens. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 7683–7694. doi:10.18653/v1/2025.acl-long.379

[24] R. Penrose. 1955. A generalized inverse for matrices. *Proceedings of the Cambridge Philosophical Society* 51, 3 (1955), 406–413. doi:10.1017/S0305004100030784

[25] Kirk Roberts, Tasmeer Alam, Steven Bedrick, Dina Demner-Fushman, Kyle Lo, Ian Soboroff, Ellen Voorhees, Lucy Lu Wang, and William R Hersh. 2021. Searching for Scientific Evidence in a Pandemic: An Overview of TREC-COVID. arXiv:2104.09632 [cs.IR]

[26] Jacob Mitchell Springer, Suhas Kotha, Daniel Fried, Graham Neubig, and Aditi Raghunathan. 2025. Repetition Improves Language Model Embeddings. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=Ahlrf2HGJR

[27] Jianlin Su, Jiarun Cao, Weijie Liu, and Yangyiwen Ou. 2021. Whitening Sentence Representations for Better Semantics and Faster Retrieval. arXiv:2103.15316 [cs.CL] https://arxiv.org/abs/2103.15316

[28] Qwen Team. 2024. Qwen2.5: A Party of Foundation Models. https://qwenlm.github.io/blog/qwen2.5/

[29] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*. https://openreview.net/forum?id=wCu6T5xFjeJ

[30] Raghuveer Thirukovalluru and Bhuwan Dhingra. 2025. GenEOL: Harnessing the Generative Power of LLMs for Training-Free Sentence Embeddings. In *Findings of the Association for Computational Linguistics: NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 2295–2308. doi:10.18653/v1/2025.findings-naacl.122

[31] Henning Wachsmuth, Shahbaz Syed, and Benno Stein. 2018. Retrieval of the Best Counterargument without Prior Topic Knowledge. In *ACL*.

[32] David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. 2020. Fact or Fiction: Verifying Scientific Claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Online, 7534–7550. doi:10.18653/v1/2020.emnlp-main.609

[33] Maurice Weber, Daniel Fu, Quentin Anthony, Yonatan Oren, Shane Adams, Anton Alexandrov, Xiaozhong Lyu, Huu Nguyen, Xiaozhe Yao, Virginia Adams, Ben Athiwaratkun, Rahul Chalamala, Kezhen Chen, Max Ryabinin, Tri Dao, Percy Liang, Christopher Ré, Irina Rish, and Ce Zhang. 2024. RedPajama: an Open Dataset for Training Large Language Models. arXiv:2411.12372 [cs.CL] https://arxiv.org/abs/2411.12372

A Details of the Main Experimental Setup

In this section, we provide additional details about the experimental setups discussed in Section 5. We evaluate all tasks from MTEB, including semantic textual similarity (STS.), classification (Class.), clustering (Cluster.), pair classification (PairClass.), re-ranking (Rerank.), retrieval (Retr.), and summarization (Sum.). Due to limited computational resources, we evaluate a subset of the retrieval tasks, consisting of eight datasets [22]: SciFact [32], ArguAna [31], NFCorpus [4], FiQA2018 [21], QuoraRetrieval [29], SCIDOCS [7], Touche2020 [3], TRECCOVID [25]. Finally we use the metrics recommended by MTEB, showing in Table 7, where the Spearman's correlation is calculated on cosine similarity. For EmbedFilter used on Mistral-7B-Instruct-V0.3, we offset the whole indices until $l_\tau = 128$. We provide the actual prompts used for PromptEOL and ECHO across different models below; "text" denotes the sentences to be embedded.

PromptEOL

Qwen

Summarize the sentence: "{text}" in one word:"

Llama

Summarize the sentence: "{text}" in one word:"

Mistral

This sentence: "{text}" means [MASK]

ECHO

Qwen

Rewrite the following paragraph: {text}. The rewritten paragraph: {text}

Llama

Rewrite the following paragraph: {text}. The rewritten paragraph: {text}

Mistral

Rewrite the following paragraph: {text}. The rewritten paragraph: {text}

Table 7: Evaluation metrics used for MTEB tasks.

Task Category	Evaluation Metric
STS	Spearman's correlation
Classification	Accuracy
Clustering	V-measure
Pair Classification	Average Precision (AP)
Reranking	Mean Average Precision (MAP)
Retrieval	nDCG@10
Summarization	Spearman's correlation

B Equivalence Transformation Proof

In the main text, we define the projection matrix as:

$$\Phi_\tau = \boldsymbol{V}[l_\tau : r_\tau]\, \boldsymbol{V}[l_\tau : r_\tau]^\top.$$

Let $\boldsymbol{V}_\tau = \boldsymbol{V}[l_\tau : r_\tau]$ for simplicity, we seek to prove the identity:

$$\|\boldsymbol{x}\,\boldsymbol{\Phi}_{\boldsymbol{\tau}}^\top - \boldsymbol{y}\,\boldsymbol{\Phi}_{\boldsymbol{\tau}}^\top\|_2 = \|\boldsymbol{x}\,\boldsymbol{V}_\tau - \boldsymbol{y}\,\boldsymbol{V}_\tau\|_2.$$

Let $\boldsymbol{z} = \boldsymbol{x} - \boldsymbol{y}$. The left-hand side can be written as:

$$\|\boldsymbol{x}\,\boldsymbol{\Phi}_{\boldsymbol{\tau}}^\top - \boldsymbol{y}\,\boldsymbol{\Phi}_{\boldsymbol{\tau}}^\top\|_2 \;=\; \|\boldsymbol{z}\,\boldsymbol{\Phi}_{\boldsymbol{\tau}}^\top\|_2 \;=\; \|\boldsymbol{z}\,\boldsymbol{V}_\tau\,\boldsymbol{V}_\tau^\top\|_2,$$

considering that $\boldsymbol{V}_\tau\,\boldsymbol{V}_\tau^\top$ is identity, thus we have:

$$\|\boldsymbol{z}\,\boldsymbol{V}_\tau\,\boldsymbol{V}_\tau^\top\|_2 \;=\; \|\boldsymbol{z}\|_2 \;=\; \|\boldsymbol{x} - \boldsymbol{y}\|_2.$$

this completes the proof of the identity in equation 1.

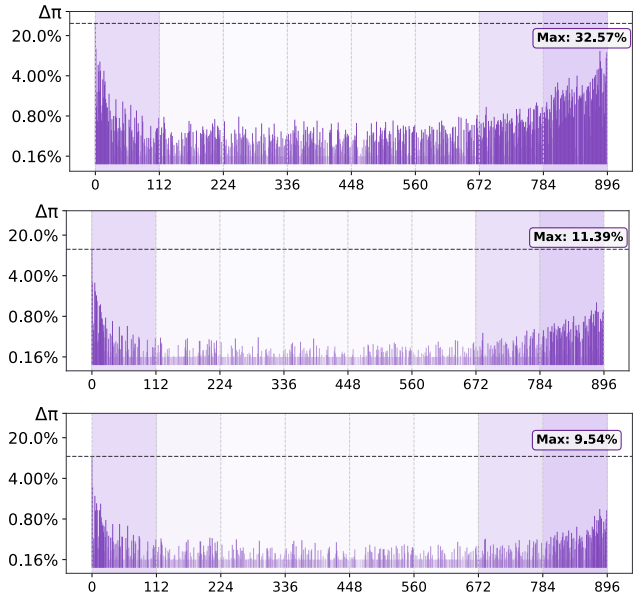


Figure 4: $\Delta\pi$ distribution for high-frequency, low-frequency and randomly sampled tokens on the Qwen model.